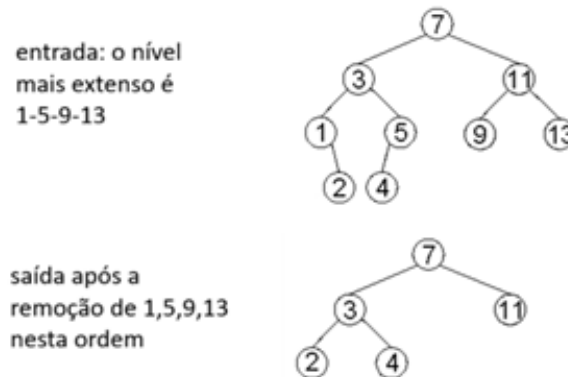


ACH2023 ALGORITMOS E ESTRUTURAS DE DADOS I

Semestre 2052-2 - Exercício prático 2 – Árvore de busca binária – t.02

Descrição do EP: A partir do modelo disponibilizado no e-disciplinas (*ep2-modelo.txt*), implementar as seguintes operações para modificar uma árvore de busca binária (não necessariamente AVL).

1. O objetivo do trabalho é implementar de forma correta e completa uma função que recebe como entrada uma árvore de busca binária (i.e., não precisa ser AVL) e faz a modificação ilustrada no exemplo a seguir.



O objetivo da função é remover o nível mais extenso, ou seja, com maior número de elementos. Havendo empate, o nível mais próximo da raiz deve ser excluído.

Note que a ordem de exclusão deve ser da esquerda para a direita, e considerando os elementos originais do nível mais extenso antes de fazer a primeira exclusão. Ou seja, mesmo que uma exclusão altere os elementos daquele nível, você deve prosseguir excluindo os elementos originais daquele nível, e não os novos elementos que possam ter ocupado aquela posição.

4. A assinatura da função é a seguinte. A função recebe como entrada uma ABB apontada por **raiz* contendo chaves inteiras, e deve retornar o ponteiro da raiz da mesma ABB após a modificação solicitada.

NO* RemoverNivelMaisExtenso(NO* raiz)

5. RESTRIÇÕES DE IMPLEMENTAÇÃO:

- a. Não use nenhum tipo de implementação estática (vetores, listas sequenciais etc.) no seu trabalho. Se necessitar de estruturas auxiliares, use sempre listas ligadas de implementação dinâmica. Uma declaração *typedef NOL* foi incluída no código exemplo caso queira criar algum tipo de lista linear.
 - b. Não use variáveis globais. A função implementada deve definir localmente todas as variáveis e estruturas auxiliares, ou chamar funções auxiliares que o façam também em um escopo local.
 - c. Não utilize código disponibilizado *online* ou em outras fontes, nem ferramentas de geração de código automático, nem ChatGPT e similares.
 - d. Não exiba nenhuma mensagem na tela, nem solicite que o usuário pressione nenhuma tecla etc. Apenas implemente a função solicitada.
6. A função *main()* serve apenas para seus testes particulares, e não deve ser entregue.
 7. Seu programa será corrigido de forma *automática*, e por isso você não pode alterar as assinaturas da função solicitada, nem os tipos de dados ou especificações (*typedef*) do modelo fornecido.
 8. O EP pode ser desenvolvido individualmente ou pelas mesmas duplas do EP1. Novas duplas não são aceitas.
 9. Não tente emprestar sua implementação para outros colegas ou duplas, nem copiar deles, pois isso invalida o trabalho de todos os envolvidos. Não há exceções a esta regra, e não será feito julgamento de mérito.

10. O programa deve ser compilável no ambiente Windows 11 com DevC++ (mas sugere-se primeiro desenvolver em uma IDE mais funcional como Codeblocks). Se você não conseguiu fazer o programa compilar sem erros dentro do prazo, não entregue. Não é previsto nenhum tipo de avaliação além dos testes de execução, o que pressupõe um programa compilável e funcional.

11. Programadores JAVA, cuidado: não existe inicialização automática de variáveis em C.

12. O que/como entregar:

- A entrega é feita por apenas um dos integrantes do grupo.
- Preencha as funções `nroUSP1`, `nroUSP2` e grupo do código disponível.
- A entrega será via upload no sistema **antes** do prazo limite da atividade.
- Entregue apenas o código da função principal e das funções auxiliares que ela invoca.
- Não entregue o código que usou na função `main()`.
- **O nome do arquivo deve ser NN.txt onde NN é o nro. do seu grupo, com dois dígitos. Favor não compactar.**

Prazos etc.:

O EP deve ser depositado no prazo definido na atividade cadastrada no sistema. Não serão aceitos EPs entregues depois do prazo, independentemente do motivo. Entregas no último dia são assim por conta e risco do aluno, e nenhum tipo de imprevisto de última hora (e.g., problemas de saúde, indisponibilidade de rede etc.) pode ser usado como justificativa para o atraso. O EP é uma atividade para ser desenvolvida ao longo de várias semanas, não no último dia da entrega.

É responsabilidade do aluno que fez o *upload* do arquivo verificar se o mesmo foi corretamente recebido pelo sistema. Atrasos/falhas na submissão invalidam o trabalho realizado. Certifique-se também de que a versão entregue é a correta. Não serão aceitas substituições.

CrITÉrios de avaliação:

A função será testada com uma série de chamadas repetidas e consecutivas, fornecendo diversas árvores não vazias contendo inteiros positivos como entrada. É assim importante assegurar que o seu programa funciona desta forma (por exemplo, chamando-o dentro de um laço *for*), e não apenas para um teste individual. Um teste é considerado correto se o resultado da soma for exatamente como o esperado, ou incorreto em caso contrário. Erros de alocação de memória ou compilação invalidam o teste, assim como a ausência de funções auxiliares necessárias para a execução do programa.

Este EP deve ser desenvolvido obrigatoriamente por *todos* os alunos de AED1. Sua nota é parte integrante da 3ª. avaliação e *não* é passível de substituição.